

Network Emulation with tc

Custom OS images · Traffic Control · Hands-on Containerlab exercises



1

Custom OS Images

Why alpine:latest isn't enough — and how to fix it

Why We Need a Custom Image

alpine:latest gives you a tiny OS — but it is missing almost every networking tool you need.

✗ bare alpine:latest

tcpdump	Packet capture — not installed
iperf3	Bandwidth measurement — missing
traceroute	Path tracing — missing
tc	Traffic control — missing (iproute2)
bash	Only /bin/sh, no bash
dig / nslookup	DNS tools — missing

✓ alpine-nettools (our custom image)

● bash	Full shell — scripts, tab completion
● iproute2	ip, ss, tc — the full toolkit
● tcpdump	Live packet capture
● iperf3	TCP/UDP throughput testing
● traceroute	Hop-by-hop path tracing
● bind-tools	dig, nslookup for DNS queries
● curl / wget	HTTP testing
● net-tools	ifconfig, netstat (legacy compat)

Building alpine-nettools – The Dockerfile

```
# Dockerfile
FROM alpine:latest

RUN apk update && apk add --no-cache \
    bash           \ # full shell
    iproute2       \ # ip, ss, tc
    iputils        \ # ping (modern)
    tcpdump        \ # packet capture
    traceroute     \ # path tracing
    iperf3         \ # bandwidth tests
    net-tools      \ # ifconfig, netstat
    bind-tools     \ # dig, nslookup
    curl wget      \ # HTTP tools
```

FROM alpine:latest

Start from the official Alpine base — tiny, secure, and fast to pull. Our additions cost only ~35 MB extra.

apk add --no-cache

Alpine's package manager. --no-cache skips the local index cache, keeping the image size minimal.

iproute2 is the key package

This single package gives you 'ip', 'ss', AND 'tc'. Without it you cannot do any traffic shaping.

Build and test:

```
docker build -t alpine-nettools:latest .
docker run --rm -it alpine-nettools:latest sh
/ # tc -V           # tc utility, iproute2-6.17.0
/ # iperf3 --version # iperf 3.19.1
```

Referencing Your Custom Image in Containerlab YAML

Before — bare alpine

```
topology:
  nodes:
    client:
      kind: linux
      image: alpine:latest # ← no tc, no
iperf3
      exec:
        - ip addr add 10.0.1.1/24 dev eth1
```

After — nettools image

```
topology:
  nodes:
    client:
      kind: linux
      image: alpine-nettools:latest # ← full
      toolkit
      exec:
        - ip addr add 10.0.1.1/24 dev eth1
```

Three ways to supply the image:

Local build (workshop)

```
docker build -t alpine-
nettools:latest .
```

Build once on the host. Containerlab picks it up automatically — no registry needed.

Pre-built registry image

```
image:
ghcr.io/myorg/nettools:latest
```

Push to Docker Hub or GHCR. Useful when every trainee needs the same image without building locally.

Inline exec install

```
exec:
  - apk add --no-cache tcpdump
```

Install packages at startup via exec. Slower (downloads every deploy) but requires no custom image.



2

Traffic Control – tc

Simulating real-world network conditions in software

What is tc? – The Linux Traffic Control Subsystem

tc is the command-line interface to the Linux kernel's built-in traffic control (QoS) engine. It can artificially add latency, loss, corruption, reordering, and bandwidth limits to any network interface.



Why do we need it?

A Containerlab virtual link is essentially a loopback — zero latency, zero loss, infinite bandwidth. Real networks are nothing like that. tc lets us inject realistic impairments so our experiments actually test protocol behaviour.



Latency

Add a fixed delay (e.g. 50 ms) or a variable jitter (± 10 ms) to emulate WAN links, satellite, or congested paths.



Packet Loss

Drop a percentage of packets randomly (e.g. 5%) or in bursts to simulate noisy wireless or failing hardware.



Bandwidth

Cap the throughput on a link to simulate DSL, 4G, or a metered WAN circuit. Uses TBF queueing disciplines.

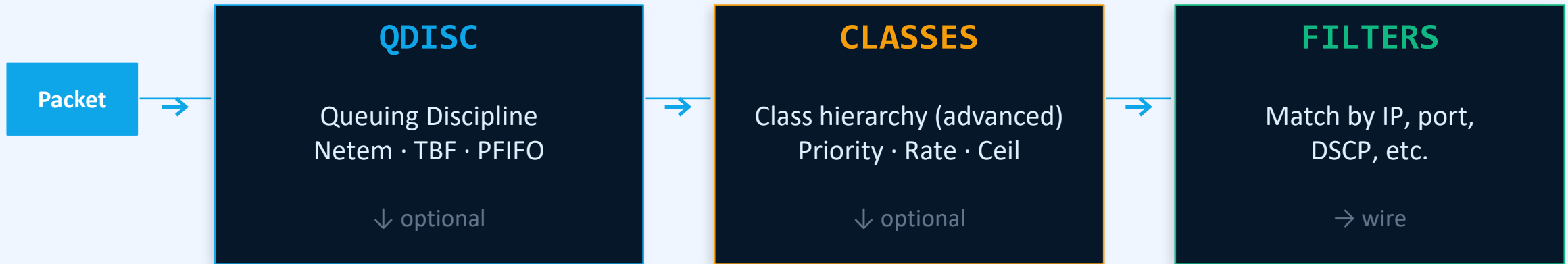


Reorder / Corrupt

Deliver packets out-of-order or flip random bits — useful for testing TCP recovery and checksum handling.

The tc Architecture – qdiscs, Classes, and Filters

tc builds a processing pipeline on each interface. Understanding this three-layer model makes every tc command make sense.



👉 For simple emulation (latency, loss, bandwidth on a whole interface) you only need the qdisc layer — one 'tc qdisc add' command is all it takes.

Simple emulation — netem / tbf

One qdisc on an interface. 'tc qdisc add dev eth1 root netem delay 50ms'. Done. No classes or filters needed.

Per-class bandwidth (advanced)

Classful qdiscs let you divide bandwidth into multiple traffic classes. Covered in a later module.

Stacked qdiscs — TBF + netem

You can chain TBF (rate limit) as root with netem as a child leaf. This gives both a bandwidth cap and artificial latency on the same interface.

tc netem – Emulating Latency, Jitter, and Packet Loss

netem (Network Emulator) is the qdisc you'll reach for most often. It runs in the kernel and has zero CPU cost at normal lab scales.

Latency & Jitter



```
# Add a fixed 50 ms one-way delay on eth1
tc qdisc add dev eth1 root netem delay 50ms

# Add 50 ms delay ± 10 ms uniform jitter
tc qdisc add dev eth1 root netem delay 50ms 10ms

# Add 50 ms delay ± 10 ms with normal distribution
tc qdisc add dev eth1 root netem delay 50ms 10ms distribution normal

# Change the delay without removing first
tc qdisc change dev eth1 root netem delay 100ms
```

Packet Loss



```
# Drop 5 % of packets randomly (uniform)
tc qdisc add dev eth1 root netem loss 5%

# Combine: 50 ms delay + 2% random loss
tc qdisc add dev eth1 root netem delay 50ms loss 2%

# Remove ALL tc rules from eth1
tc qdisc del dev eth1 root
```

Why ±jitter matters

Real WAN links don't have a fixed delay. Adding $\pm 10\text{ms}$ jitter means each packet arrives at a slightly different time — just like on the internet. This reveals how well applications handle out-of-order delivery.

tc qdisc vs tc qdisc change

Use 'add' the first time you apply a rule. Use 'change' to modify an existing qdisc in place without disrupting ongoing flows. 'del' removes the qdisc completely.

Delay is one-way!

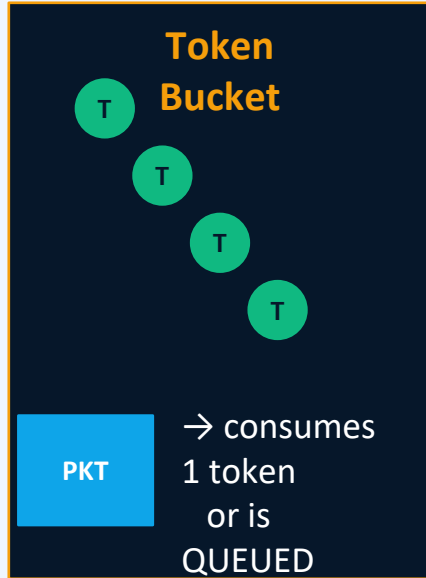
50ms added on eth1 of the router means 50ms in that one direction only. For round-trip symmetry, add the same rule on the return interface too (or add it on both endpoints).

Combining params

You can combine delay, loss, corrupt, reorder, and duplicate in a single command. All parameters are applied in the order they appear.

TBF – Token Bucket Filter: Simple Bandwidth Limiting

TBF is the go-to qdisc for capping the total throughput of a single interface to a fixed rate.



Tokens refill at exactly the rate you set.
Packets can only exit when a token is available.

TBF command syntax:

```
# Limit eth1 to 10 Mbit/s
tc qdisc add dev eth1 root tbf \
    rate 10mbit \           # the sustained throughput cap
    burst 32kbit \          # max burst size (tokens in bucket)
    latency 50ms            # max time a packet waits in queue

# Limit to 1 Mbit/s (DSL simulation)
tc qdisc add dev eth1 root tbf \
    rate 1mbit burst 32kbit latency 100ms

# Combine TBF (rate limit) + netem (latency)
# Use an intermediate queueing discipline (handle):
tc qdisc add dev eth1 root handle 1: tbf \
    rate 10mbit burst 32kbit latency 50ms
tc qdisc add dev eth1 parent 1:1 handle 10: \
    netem delay 20ms loss 1%
```

rate	The sustained throughput cap. Examples: 1mbit, 100kbit, 500kbit, 10mbit, 1gbit
burst	How many bytes can be sent at full wire speed before the rate kicks in. Usually 32kbit–64kbit.
latency	Maximum time a packet sits in the TBF queue before being dropped. Acts as a buffer limit.

Verifying Your tc Rules – How to Confirm They Work

After applying tc rules, always verify the effect with measurement tools. The rule may look correct but have a typo in the unit.

① List active rules

Always start by inspecting what is currently applied:

```
# Show qdisc on eth1
tc qdisc show dev eth1

# Show classes (classful qdiscs only)
tc class show dev eth1

# Show filters
tc filter show dev eth1
```

If the output shows 'qdisc noqueue' or just the default 'pfifo_fast', no custom rules are active.

② Verify latency with ping

Compare RTT before and after applying netem:

```
# Baseline (before tc)
ping -c 10 10.0.2.1
# rtt avg = 0.28 ms

# Apply 50ms delay on router eth1
tc qdisc add dev eth1 root \
    netem delay 50ms

# Re-test (should see ~50ms RTT)
ping -c 10 10.0.2.1
# rtt avg = 50.31 ms ✓
```

The RTT increase equals the one-way delay you set. For 50ms netem delay expect ~50ms RTT increase on a local network.

③ Verify bandwidth — iperf3

Run iperf3 server on one node, client on another:

```
# Node: server – start listener
iperf3 -s

# Node: client – run 10s test
iperf3 -c 10.0.2.1 -t 10

# With TBF 1mbit limit applied:
# [SUM] ... 1.01 Mbits/sec ✓

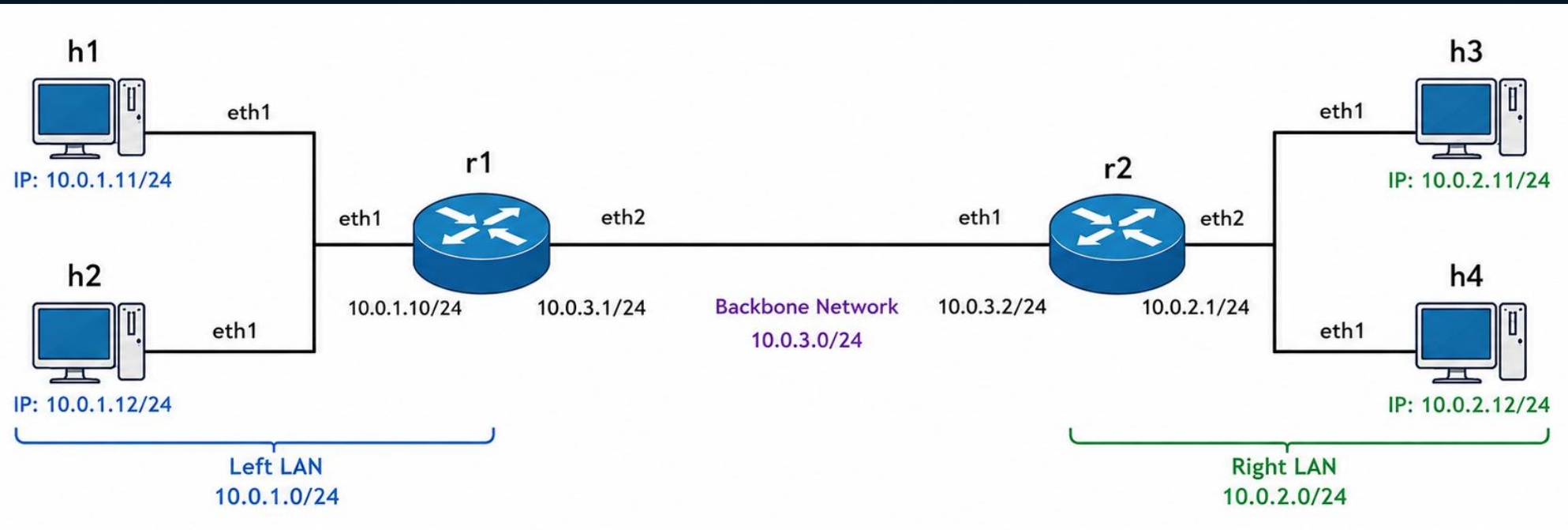
# UDP test (shows packet loss %):
iperf3 -c 10.0.2.1 -u \
    -b 5M -t 10
```

UDP mode (-u) reports the actual loss percentage — perfect for verifying a netem loss rule.

tc Quick-Reference Cheatsheet

<code>tc qdisc show dev ETH</code>	List active qdiscs on interface ETH
<code>tc qdisc del dev ETH root</code>	Remove ALL tc rules from interface ETH
<code>tc qdisc add dev ETH root netem delay Xms</code>	Add X ms one-way latency
<code>tc qdisc add dev ETH root netem delay Xms Yms</code>	Add X ms delay ± Y ms jitter
<code>tc qdisc add dev ETH root netem loss X%</code>	Drop X% of packets randomly
<code>tc qdisc add dev ETH root netem delay Xms loss Y%</code>	Combine: latency + loss in one rule
<code>tc qdisc change dev ETH root netem delay Xms</code>	Modify existing netem rule in-place
<code>tc qdisc add dev ETH root tbf rate Xmbit burst 32kbit latency 50ms</code>	Limit throughput to X Mbit/s (TBF)
<code>tc qdisc add dev ETH root handle 1: tbf rate Xmbit burst 32kbit latency 50ms</code>	TBF with handle (for chaining)
<code>tc qdisc add dev ETH parent 1:1 handle 10: netem delay 20ms</code>	Chain netem under TBF — rate + latency together
<code>ping -c 20 IP</code>	Verify latency — compare avg RTT before/after
<code>iperf3 -s / iperf3 -c IP -t 10</code>	Verify bandwidth — server then client
<code>iperf3 -c IP -u -b 5M -t 10</code>	Verify loss — UDP mode reports loss %

Assignment – tc on the Dumbbell Topology



1 Deploy with custom image

Rewrite your dumbbell YAML so every node uses `alpine-nettools:latest`. Deploy and confirm all nodes are running with `clab inspect`.

2 Baseline measurements

From h1, ping h3 and record the avg RTT. Then run `iperf3` from h1 to h3 (start `iperf3 -s` on h3 first). Record the baseline throughput in Mbit/s.

3 Emulate a WAN backbone

On r1's eth2 (backbone-facing), apply: 50ms delay + 1% loss + 10mbit rate cap. Command:
`tc qdisc add dev eth2 root handle 1: tbf rate 10mbit burst 32kbit latency 100ms`
`tc qdisc add dev eth2 parent 1:1 handle 10: netem delay 50ms loss 1%`

4 Re-measure and compare

Repeat the ping and `iperf3` tests. Ping RTT should increase by ~50ms. `iperf3` throughput should drop to ~10mbit. UDP `iperf3` should show ~1% loss.

5 Verify with tcpdump

While h1 pings h3, run `tcpdump` on r2's eth1. You should see ICMP packets arriving at the rate-limited, delayed pace. Count the inter-packet gap.

6 Clean up and explain

Remove the `tc` rules with `'tc qdisc del dev eth2 root'`. Re-run `iperf3` and confirm throughput returns to baseline. Be prepared to explain what TBF and netem each contributed.

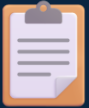
Summary – What We Covered



Build a custom alpine-nettools image with Dockerfile — one RUN line installs all tools including iproute2 (which provides tc).



TBF caps a single interface to a fixed rate. Simple, predictable, best for most exercises.



Reference it in any YAML with 'image: alpine-nettools:latest'. No other change needed.



TBF + netem can be chained: use a handle on TBF and attach netem as a child for combined rate limiting and latency.



tc netem adds latency, jitter, and loss. One command, zero cost, instant effect.



Always verify: 'tc qdisc show' to read rules, ping for latency, iperf3 for throughput and loss.